Plugin Developer Guide – Fighting Engine

Plugins let you add custom JavaScript code to the game without modifying the core engine. You can use them to:

- Add new game modes
- Modify fighter behavior on the fly
- Create custom HUD overlays
- Log match statistics
- Integrate with external APIs
- Automate testing or AI

Where to Find the Plugin Editor

1. Open the **Settings** modal (click ⚙ Settings in the top menu).
2. Scroll down to the ✒ **Custom Plugins** section.

You will see a list of already added plugins, a file upload field, and a button to paste code manually.

➕ Adding a Plugin

Option 1 – Upload a .js file

- Click the **Choose File** button under .js file upload.
- Select a valid JavaScript file.
- The plugin will be added automatically with the filename (without extension) as its name.

Option 2 – Paste code manually

- Click the ➕ **Enter custom code** button.
- A prompt will ask for a plugin name (e.g., *"My Custom AI"*).
- A second prompt will ask for the JavaScript code.
- Paste your code and confirm.

After adding, the plugin appears in the list with a toggle (▶/⏸) and a delete button.

Enabling / Disabling Plugins

- Use the ▶/⏸ button next to each plugin to enable or disable it.
- Disabled plugins are **not executed** when settings are saved.

Writing a Plugin – The API

When you save your settings (by clicking 💾 **Save** in the Settings modal),
all **enabled** plugins are executed immediately via eval().
Plugins run in the global scope, so they have access to everything in the page.

Global Objects & Functions Available

Object / Function	Description
project	The main data object (characters, stages, settings)
p1, p2	The current Fighter instances (may be null if not initialized).
camera	Object with zoom, offsetX, offsetY.
projectiles, mines	Arrays of active projectiles/mines.
springPlatforms	Array of platform objects.
activeFallingItems	Array of falling items.
fullReset(), resetRound(), setMode(), setStage(), etc.	All core game functions.
showToast(msg, type)	Show a message on screen.
playSystemSound(type)	Play a system sound.
localStorage	Browser storage.
console	Debug logging.

Important: Plugins are **evaluated once** when settings are saved.
If you need code to run every frame, you must set up your own setInterval or hook into the game loop (e.g., override update or render).
Be careful not to break the game – test your plugins in a safe environment.

Example Plugin – Auto-Heal Every 5 Seconds

javascript

```
// This plugin heals both fighters by 10 HP every 5 seconds
console.log('Auto-heal plugin loaded!');
```

```
setInterval(() => {
    if (p1 && p1.health > 0) {
        p1.health = Math.min(p1.maxHealth, p1.health + 10);
    }
    if (p2 && p2.health > 0) {
        p2.health = Math.min(p2.maxHealth, p2.health + 10);
    }
    showToast('💖 Healed!', 'info');
}, 5000);
```

Example Plugin – Custom Game Mode (Sudden Death)

javascript

```
// When health drops below 20%, increase damage by 50%
console.log('Sudden Death plugin loaded!');

// Override the takeDamage method for both fighters
const originalTakeDamage = Fighter.prototype.takeDamage;

Fighter.prototype.takeDamage = function(dmg, fromDir, blocked) {
    if (this.health < this.maxHealth * 0.2 && dmg > 0) {
        dmg = Math.floor(dmg * 1.5); // +50% damage
    }
    originalTakeDamage.call(this, dmg, fromDir, blocked);
};
```

Tip: You can also store state inside the plugin using closures or global variables.

Example Plugin – Log Match Results to Console

javascript

```
// Log who wins each round
console.log('Match logger plugin loaded!');
```

```
const originalHandleRoundEnd = window.handleRoundEnd; // store original


window.handleRoundEnd = function(winner) {
    console.log(`🏆 Round ended! Winner: ${winner}`);
    if (winner === 'p1') console.log('P1 wins the round');
    else if (winner === 'p2') console.log('P2 wins the round');
    else console.log('Draw!');
    originalHandleRoundEnd(winner);
};
```

Testing Your Plugin

1. Write your plugin code.
2. Add it via the plugin editor.
3. Save settings – your plugin will be executed.
4. Start a match and observe the effects.

If something goes wrong, check the browser console (F12) for errors.
You can disable the plugin, edit the code, and save again to re-apply.

Advanced Usage – Creating Reusable Plugin Libraries

You can write plugins that export functions or classes and then use them in other plugins. Since all plugins run in the same global scope, you can share data by attaching properties to the window object.

Example:

javascript

```
// Plugin 1 – utility library
window.myUtils = {
```

```
    randomDamage(min, max) {
        return Math.floor(Math.random() * (max - min + 1)) + min;
    }
};
```

javascript

```
// Plugin 2 – uses the library
console.log('Random damage:', myUtils.randomDamage(5, 20));
```

Plugin Lifecycle

Event	Action
Settings saved	All enabled plugins are executed via eval().
Match reset	Plugins **are not** re-executed; they persist.
Page reload	Plugins are lost unless saved in the project export.

✅ **Best practice**: Export your project after adding plugins to keep them.

Removing a Plugin

- Open the Settings modal.
- Find the plugin in the list and click the 🗑 delete button.
- Save settings to apply removal.

Security Note

Plugins run with **full access** to the page. Only load plugins from trusted sources.
When publishing your game, plugins are included in the exported HTML.